

Développement d'application Web

Site Web transactionnel

Table des matières

Interaction avec MySQL.....	2
Création d'une base de données.....	2
Accès à une base de données via PHP.....	3
Cookies.....	5
Création d'un cookie.....	5
Lecture d'un cookie.....	6
Authentification.....	7
Authentification par http.....	7
Formulaire d'authentification.....	8
Stockage des mots de passe.....	10
Ajout de compte utilisateurs.....	10
Session.....	11
Sécurité et authentification.....	11
Panier d'achat.....	14
Références.....	16

Document écrit par Stéphane Gill

Mars 2020



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

Une page Web dynamique est une page web générée à la demande, par opposition à une page web statique. Le contenu d'une page web dynamique peut donc varier en fonction d'informations (heure, nom de l'utilisateur, formulaire rempli par l'utilisateur, etc.) qui ne sont connues qu'au moment de sa consultation. À l'inverse, le contenu d'une page web statique est a priori identique à chaque consultation. Un site Web transactionnel est un site Web composé de pages Web dynamiques capables de contrôler des transactions en ligne avec des parties authentifiées.

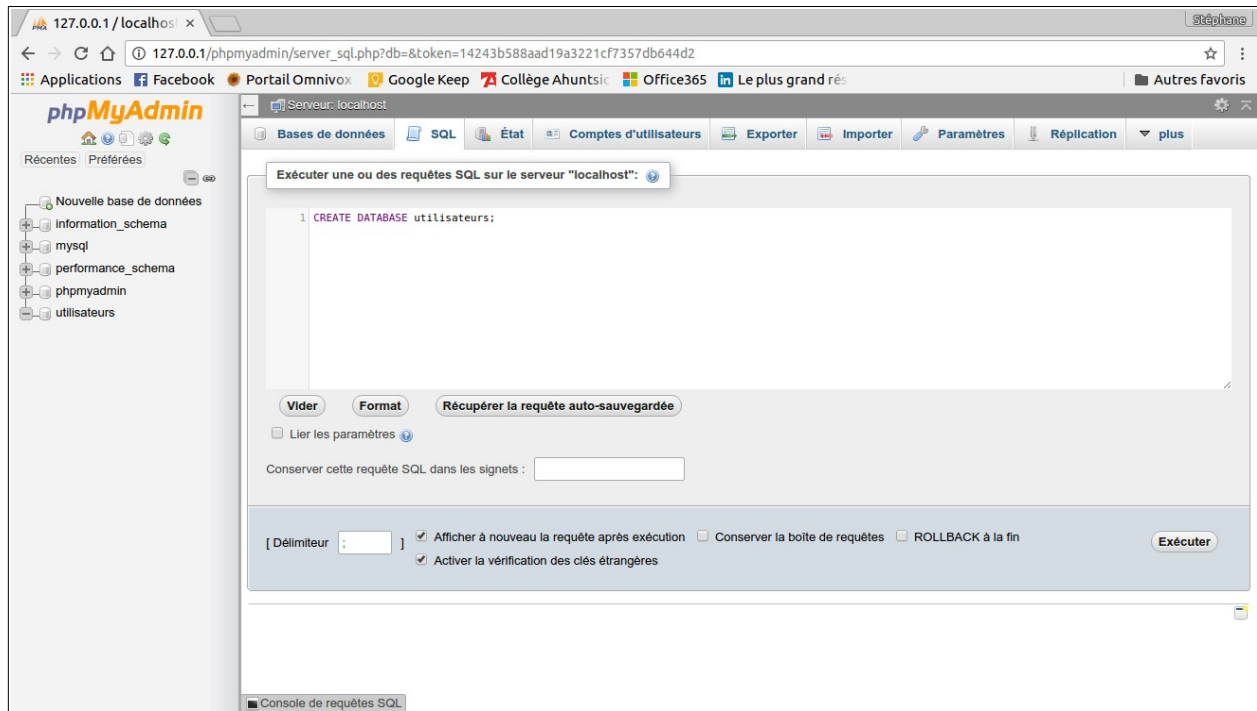
Dans ce chapitre, l'utilisation des cookies, des sessions, de l'authentification et des bases de données est abordé.

Interaction avec MySQL

MySQL est un système de gestion de bases de données relationnelles (SGBDR). Il est un des SGBDR les plus utilisés au monde, autant par le grand public que par des professionnels. MySQL fonctionne sur de nombreux systèmes d'exploitation différents, incluant AIX, IBM i-5, BSDi, FreeBSD, HP-UX, Linux, Mac OS X, NetWare, NetBSD, OpenBSD, OS/2 Warp, SGI IRIX, Solaris, SCO OpenServer, SCO UnixWare, Tru64 Unix, Windows. Il est accessibles en utilisant les langages de programmation C, C++, VB, VB .NET, C#, Delphi/Kylix, Eiffel, Java, Perl, PHP, Python, Windev, Ruby et Tcl ; une API spécifique est disponible pour chacun d'entre eux.

Création d'une base de données

La création d'une base de données peut s'effectuer via *phpMyAdmin*. Il est possible d'accéder *phpMyAdmin* via l'URL suivant : <http://127.0.0.1/phpmyadmin>.



Exemple 1 : Créer une base de données avec une table

```

1 CREATE DATABASE monBlogue;
2
3 USE monBlogue;
4
5 CREATE TABLE utilisateurs (
6   prenom VARCHAR(32) NOT NULL,
7   nom VARCHAR(32) NOT NULL,
8   username VARCHAR(32) NOT NULL UNIQUE,
9   password VARCHAR(32) NOT NULL
10  );
11
12 INSERT INTO `utilisateurs` (`prenom`, `nom`, `username`, `password`)
13   VALUES ('Stephane', 'Gill', 'gills', 'qwerty');

```

Il est aussi possible d'exporter et d'importer une base de données.

Accès à une base de données via PHP

Une Interface de programmation d'application (API), définit les classes, méthodes, fonctions et variables dont votre application va faire usage pour exécuter différentes tâches. Dans le cas des applications PHP, les communications avec une base de données se font généralement par une API qui est disponible dans une extension PHP.

Les API peuvent être procédurales ou orientées objet. Avec une API procédurale, vous pouvez appeler les fonctions pour exécuter des commandes. Tandis qu'avec une API orientée objet, vous devez créer des objets et appeler les méthodes de ces objets. Des deux, c'est la dernière qui est généralement préférée, car elle est plus moderne et conduit à du code plus organisé.

Il y a trois API principales pour se connecter à MySQL :

- L'extension *mysql*
- L'extension *mysqli*
- *PHP Data Objects* (PDO)

Chacune a ses avantages et ses inconvénients. Dans cette section, l'extension *PDO* est présentée, elle est incluse dans PHP depuis la version 5.

Exemple 2 : Accéder à la base de données « monBlogue »

```
1  <?php
2  $servername = "localhost";
3  $username = "root";
4  $password = "";
5  $dbname = "monBlogue";
6  $charset = "utf8mb4";
7
8  $dsn = "mysql:host=$servername;dbname=$dbname;charset=$charset";
9  $options = [
10     PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
11     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
12     PDO::ATTR_EMULATE_PREPARES   => false,
13 ];
14
15 try {
16     $pdo = new PDO($dsn, $username, $password, $options);
17 } catch (\PDOException $e) {
18     die("Echec de la connexion: " . $e->getMessage());
19 }
20
21 echo "Connexion réussie! <br>";
22
23 $stmt = $pdo->prepare("SELECT * FROM utilisateurs");
24 $stmt->execute();
25 $result = $stmt->fetchAll();
26
27 print_r($result);
28
29 foreach ($result as $row)
30 {
31     echo "<br>username: " . $row["username"]. " - mot de passe: ".
32         $row["password"] . "<br>";
33 }
```

L'accès à MySQL se fait via la création d'un objet *PDO* (ligne 16). Le constructeur de cet objet reçoit 4 paramètres, soit :

- l'information sur la base de données;
- Le nom d'utilisateur;
- Le mot de passe;
- Les options.

Aux lignes 23 et 24, on remarque des requêtes SQL standard sont envoyées à MySQL à l'aide des méthodes `prepare()` et `execute()`. Le résultat de la requête est récupéré dans un tableau qu'il faut parcourir pour extraire l'information (ligne 29 à 33).

Cookies

Dans cette section, la création et la manipulation des cookies est présentée. Un cookie est un petit fichier texte (faisant au maximum 65 Ko) stocké sur le disque dur du visiteur du site. Ce fichier texte permet de sauvegarder diverses informations concernant ce visiteur afin de pouvoir les réutiliser lors de sa prochaine visite sur le site. Par exemple, on pourrait très bien stocker dans ce cookie le nom du visiteur et par la suite, afficher son nom à chaque fois qu'il se connectera sur le site.

Création d'un cookie

La création de cookies s'effectue grâce à la fonction `setcookie()`. L'exemple suivant présente l'utilisation de cette fonction.

Exemple 3 : Créer un cookie

```

1  <html>
2  <body>
3      <?php
4          $cookie_name = "usager";
5          $cookie_value = "Stephane";
6          setcookie($cookie_name, $cookie_value, time() + 60, "/");
7      ?>
8  <p>Ecrire un cookie!</p>
9  </body>
10 </html>
```

À la ligne 6, la fonction `setcookie()` permet d'envoyer un cookie de nom « usager » portant la valeur « Stephane ». Le troisième paramètre de cette fonction spécifie la « fin de vie » du cookie, soit 60

secondes après sa création. Habituellement, la durée de vie d'un cookie est beaucoup plus longue que 60 secondes. La fonction `time()` retourne le nombre de secondes écoulées depuis le 1er janvier 1970 jusqu'à l'instant présent.

Lecture d'un cookie

Dans l'exemple suivant, la valeur d'un cookie est récupéré.

Exemple 4 : Récupérer la valeur d'un cookie.

```
1 <html>
2 <body>
3 <?php
4     $cookie_name = "usager";
5     if(!isset($_COOKIE[$cookie_name])) {
6         echo "Le Cookie '" . $cookie_name . "' n'existe pas!";
7     } else {
8         echo "Le Cookie '" . $cookie_name . "' existe!<br>";
9         echo "Sa valeur est: " . $_COOKIE[$cookie_name];
10    }
11 ?>
12 </body>
13 </html>
```

La récupération de la valeur d'un cookie s'effectue via la variable globale `$_COOKIE`. À la ligne 5, la fonction `isset()` permet de s'assurer de l'existence du cookie. En revanche, l'envoi d'un cookie ayant la même valeur pour tous les visiteurs d'un site, n'est pas toujours approprié.

Exemple 5 : Créer un cookie avec le nom du visiteur su site Web (`pseudoCookie.php`)

```
1 <html>
2 <head>
3 <title>Index du site</title>
4 <body>
5
6 <?php
7 if (isset($_COOKIE['pseudo'])) {
8     echo 'Bonjour ' . $_COOKIE['pseudo'] . ' !';
9 }
10 else {
11     echo 'Notre cookie n\'est pas déclaré.';
12     echo '<form action="traitement.php" method="post">';
13     echo 'Votre nom : <input type = "texte" name = "nom"><br />';
14     echo '<input type = "submit" value = "Envoyer">';
15 }
16 ?>
17
18 </body>
19 </html>
20
```

À la ligne 7, l'existence du cookie est vérifié, s'il n'existe pas, un formulaire html est affiché. Ce formulaire permet de saisir le nom du visiteur du site. Cette information est envoyée à un script PHP qui va créer le cookie contenant le nom du visiteur.

Exemple 6 : Script de création du cookie (traitement.php)

```
1 <?php
2 If (isset($_POST['nom'])) {
3     setcookie ("pseudo", $_POST['nom'], time() + 60);
4     redirection ('pseudoCookie.php');
5 }
6 else {
7     echo 'La variable du formulaire n\'est pas déclarée.';
8 }
9
10 // fonction nous permettant de faire des redirections
11 function redirection($url){
12     if (headers_sent()){
13         print('<meta http-equiv="refresh" content="0;URL=' . $url . '">');
14     }
15     else {
16         header("Location: $url");
17     }
18 }
19 ?>
```

Une fois le cookie créé (ligne 3), l'utilisateur sera redirigé vers la page d'accueil (pseudoCookie.php).

Authentification

L'authentification est la procédure qui consiste, pour un système informatique, à vérifier l'identité d'une personne ou d'un ordinateur afin d'autoriser l'accès de cette entité à des ressources (systèmes, réseaux, applications...). L'authentification permet donc de valider l'authenticité de l'entité en question.

Authentification par http

L'authentification HTTP ou identification HTTP (spécifiée par RFC 26171) permet de s'identifier auprès d'un serveur HTTP en lui montrant que l'on connaît le nom d'un utilisateur et son mot de passe, afin d'accéder aux ressources à accès restreint de celui-ci.

Lorsqu'un client HTTP demande une ressource protégée au serveur, celui-ci répond de différente façon selon la requête :

- soit la requête ne contient pas d'en-tête HTTP d'identification, dans ce cas le serveur répond avec le code HTTP 401 (Unauthorized : non autorisé) et envoie les en-têtes d'information sur l'identification demandée;
- soit la requête contient les en-têtes HTTP d'identification, dans ce cas, après vérification du nom et du mot de passe, si l'identification échoue, le serveur répond par le code 401 comme dans le cas précédent, sinon il répond de manière normale (code 200 OK).

```

1  <?php
2  if (isset($_SERVER['PHP_AUTH_USER']) && isset($_SERVER['PHP_AUTH_PW']))
3  {
4      if($_SERVER['PHP_AUTH_USER'] == "Stephane" &&
5          $_SERVER['PHP_AUTH_PW'] == "allo"){
6          echo "Bienvenue Usager: " . $_SERVER['PHP_AUTH_USER'] .
7              " Mot de passe: " . $_SERVER['PHP_AUTH_PW'] . "<br>";
8      } else {
9          die ("Mot de passe invalide");
10     }
11 } else {
12     header('WWW-Authenticate: Basic realm="Restricted Section"');
13     header('HTTP/1.0 401 Unauthorized');
14     die("SVP Entrer votre nom d'utilisateur et votre mot de passe");
15 }
16 ?>

```

Les lignes 2 et 3 s'assurent que l'utilisateur a été authentifié, si ce n'est pas le cas une fenêtre d'authentification apparaît.

Formulaire d'authentification

L'authentification d'un utilisateur peut aussi se faire via un formulaire html.

Exemple 7 : Formulaire d'authentification (authForm_PDO.html)

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Exemple: formulaire d'authentification</title>
5  </head>
6  <body>
7      <form method="post" action="connect_PDO.php">
8          Nom d'utilisateur: <input type="text" name="username"/><br>
9          Mot de passe: <input type="password" name="password"/><br>
10         <input type="submit" value="Se connecter">
11     </form>
12 </body>
13 </html>

```

L'information du formulaire est envoyée au script connect.php (ligne 7).

Exemple 8 : Script PHP d'authentification (connect_PDO.php)

```
1  <?php
2  if(!isset($_POST["username"]) && !isset($_POST["password"]))
3  {
4      header("Location: authForm.html");
5      Exit;
6  }
7  else
8  {
9      $servername = "localhost";
10     $username = "root";
11     $password = "";
12     $dbname = "monBlogue";
13     $charset = "utf8mb4";
14
15     $dsn = "mysql:host=$servername;dbname=$dbname;charset=$charset";
16     $options = [
17         PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
18         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
19         PDO::ATTR_EMULATE_PREPARES   => false,
20     ];
21
22     try {
23         $pdo = new PDO($dsn, $username, $password, $options);
24     } catch (\PDOException $e) {
25         die("Echec de la connexion: " . $e->getMessage());
26     }
27
28     $Blog_user = $_POST["username"];
29     $Blog_pass = $_POST["password"];
30
31     $sql = "SELECT * FROM utilisateurs WHERE username=? AND password=?";
32
33     $stmt = $pdo->prepare($sql);
34     $stmt->execute([$Blog_user, $Blog_pass]);
35     $result = $stmt->fetchAll();
36
37     if($stmt->rowCount() == 0){
38         echo "Usager inexistant ou Mauvais mot de passe<br/>";
39     } else {
40         echo "Bon mot de passe<br/>";
41     }
42 }
43 ?>
```

Aux lignes 28 et 29, l'information reçue du formulaire est stocké dans 2 variables qui seront utilisés à la ligne 31 pour construire une requête SQL qui valide le nom de l'utilisateur et son mot de passe.

Stockage des mots de passe

Les mots de passe d'un site Web sont habituellement stocker dans une base de données MySQL. Avant de le stocker, il est préférable de l'encrypter. Plusieurs algorithmes d'encryptions sont déjà implémentés dans PHP et il suffit d'utiliser l'instruction `hash()`.

```
$Blog_hash = hash("md5", $Blog_pass);
```

Ajout de compte utilisateurs

Exemple 9 : Ajouter un compte utilisateur dans la base de données.

```
1  <?php
2  $servername = "localhost";
3  $username = "root";
4  $password = "";
5  $dbname = "monBlogue";
6  $charset = "utf8mb4";
7
8  $dsn = "mysql:host=$servername;dbname=$dbname;charset=$charset";
9  $options = [
10     PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
11     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
12     PDO::ATTR_EMULATE_PREPARES   => false,
13 ];
14
15 try {
16     $pdo = new PDO($dsn, $username, $password, $options);
17 } catch (\PDOException $e) {
18     die("Echec de la connexion: " . $e->getMessage());
19 }
20
21 $Blog_prenom = "Samuel";
22 $Blog_nom = "Gill";
23 $Blog_user = "samsam";
24 $Blog_pass = "1234";
25
26 $Blog_hash = hash("md5", $Blog_pass);
27
28 //Verifier si le username existe avant de faire l'ajout
29 $sql = "INSERT INTO utilisateurs VALUES (?, ?, ?, ?)";
30 $stmt = $pdo->prepare($sql);
31 $stmt->execute([$Blog_prenom, $Blog_nom, $Blog_user, $Blog_hash]);
32
33 ?>
```

Les instructions, des lignes 8 à 19, permettent d'établir la connexion avec la base de données. Les informations sur les utilisateurs sont stockées dans 4 variables (ligne 21 et 24). Il est important d'encrypter le mot de passe (ligne 26). Avant d'ajouter le compte utilisateur à la base de données (ligne 29) il serait préférable de vérifier l'existence du nom d'utilisateur dans la base de données.

Session

Les sessions sont un moyen simple de stocker des données individuelles pour chaque utilisateur en utilisant un identifiant de session unique. Elles peuvent être utilisées pour faire persister des informations entre plusieurs pages. Les identifiants de session sont normalement envoyés au navigateur via des cookies de session, et l'identifiant est utilisé pour récupérer les données existantes de la session. L'absence d'un identifiant ou d'un cookie de session indique à PHP de créer une nouvelle session, et génère ainsi un nouvel identifiant de session.

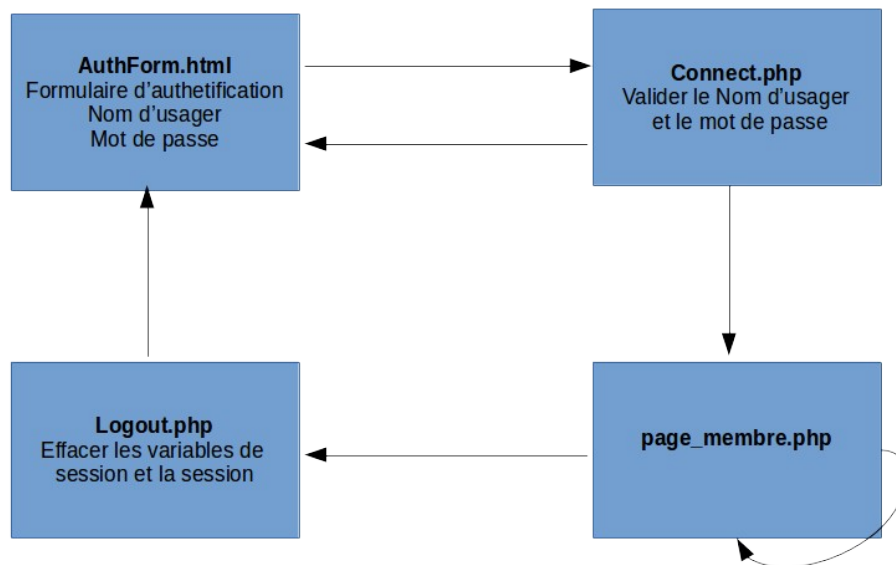
Les sessions suivent une cinématique simple. Lorsqu'une session est démarrée, PHP va soit récupérer une session existante en utilisant l'identifiant de session passé (habituellement depuis un cookie de session) ou si aucun identifiant de session n'est passé, il va créer une nouvelle session. PHP va ainsi peupler la variable superglobale `$_SESSION` avec toutes les données de session une fois la session démarrée. Lorsque PHP s'arrête, il va prendre automatiquement le contenu de la variable superglobale `$_SESSION`, le linéariser, et l'envoyer pour stockage au gestionnaire de sauvegarde de session.

Sécurité et authentification

Afin de voir concrètement comment fonctionnent les sessions, prenons alors un exemple simple :

- imaginons qu'un site possède une section membre où chaque membre devra s'authentifier avant de pouvoir y accéder.
- de plus, il faut s'assurer qu'il s'agisse toujours de ce même membre qui est connecté.

Le site web aura alors une page contenant un formulaire permettant aux visiteurs de se connecter à une section membre.



Exemple 10 : Formulaire d'authentification (authForm.html)

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Exemple: formulaire d'authentification</title>
5  </head>
6  <body>
7    <form method="post" action="connect.php">
8      Nom d'utilisateur: <input type="text" name="username"/><br/>
9      Mot de passe: <input type="password" name="password"/><br/>
10     <input type="submit" value="Se connecter">
11   </form>
12 </body>
13 </html>

```

Ce formulaire transfère son contenu (nom d'utilisateur et mot de passe) à un script php (connect . php).

Exemple 11 : Vérifier le nom d'utilisateur et le mot de passe (connect . php)

```

1  <?php
2  $user_valide = "moi";
3  $pass_valide = "lemien";
4
5  if(!isset($_POST["username"]) && !isset($_POST["password"]))
6  {
7    echo '<body onLoad="alert(\'Nom ou mot de passe manquant\')">';
8    echo '<meta http-equiv="refresh" content="0;URL=authForm.html">';
9  }
10 else
11 {
12   if ($user_valide == $_POST["username"] &&

```

```

13     $pass_valide == $_POST["password"])
14 {
15     session_start ();
16     $_SESSION["username"] = $_POST["username"];
17     $_SESSION["password"] = $_POST["password"];
18     header ('location: pageMembre.php');
19     echo "creation session";
20 }
21 else
22 {
23     echo '<body onLoad="alert(\'Membre non reconnu...\')">';
24     echo '<meta http-equiv="refresh" content="0;URL=authForm.html">';
25 }
26
27 }
28 ?>

```

À la ligne 5, l'existence des variables \$username et \$password est vérifiée. Puis, à la ligne 12, la validité du mot de passe est vérifiée. L'instruction session_start() permet l'ouverture de la session (ligne 15). Après l'ouverture de la session, les informations sur l'utilisateur sont stockées dans les variables de session puis l'utilisateur est redirigé vers la page de la section membre (ligne 18).

Exemple 12 : Page de la section membre (page_membre.php)

```

1  <?php
2  session_start ();
3
4  if (isset($_SESSION["username"]) && isset($_SESSION["password"])) {
5
6      echo '<html>';
7      echo '<head>';
8      echo '<title>Page de notre section membre</title>';
9      echo '</head>';
10
11     echo '<body>';
12     echo 'Votre login est ' . $_SESSION["username"] .
13         ' et votre mot de passe est ' . $_SESSION["password"] . ' .';
14     echo '<br/>';
15
16     echo '<a href="./logout.php">Déconnection</a>';
17 }
18 else {
19     echo 'Les variables ne sont pas déclarées.';
20 }
21 ?>

```

Une étape indispensable dans toutes les pages de la section membre est de démarrer la session (ligne 2). Puis à la ligne 4, il faut vérifier s'il s'agit bien d'un visiteur enregistré en vérifiant l'existence des variables de session.

Exemple 13 : Script de déconnexion (logout.php).

```
1 <?php
2 session_start();
3 session_unset();
4 session_destroy();
5
6 header('location: authForm.html');
7 ?>
```

À la ligne 2, la session est démarrée puis, l'instruction `session_unset()` détruit les variables de la session tandis que l'instruction `session_destroy()` détruit la session.

Panier d'achat

Une méthode simple pour créer un panier d'achat en PHP est d'utiliser les variables de sessions.

Source: <http://jcrozier.developpez.com/articles/web/panier/>

Panier
+libelleProduit: String +qteProduit: int +prixProduit: float +verrou: boolean
+ajouterArticle(): boolean +supprimerArticle(): void +modifierArticle(): void +montantGlobal(): float +supprimePanier(): void +isVerrouille(): boolean +compterArticles(): int

Exemple 14 : Création du panier

```
1 function creationPanier(){
2     if (!isset($_SESSION['panier'])){
3         $_SESSION['panier']=array();
4         $_SESSION['panier']['libelleProduit'] = array();
5         $_SESSION['panier']['qteProduit'] = array();
6         $_SESSION['panier']['prixProduit'] = array();
7         $_SESSION['panier']['verrou'] = false;
8     }
9     return true;
10 }
```

Dans un premier temps, le script vérifie l'existence du panier, si celui-ci n'existe pas, il est créé. La variable 'verrou' permet de verrouiller toute action sur le panier, le verrou est à activer lorsque vous passez votre panier en paiement.

Exemple 15 : Ajout d'un article au panier

```
1 function ajouterArticle($libelleProduit,$qteProduit,$prixProduit){
2
3     if (creationPanier() && !isVerrouille())
4     {
5         //Si le produit existe déjà on ajoute seulement la quantité
6         $positionProduit = array_search($libelleProduit,
7             $_SESSION['panier']['libelleProduit']);
8
9         if ($positionProduit !== false)
10        {
11            $_SESSION['panier']['qteProduit'][$positionProduit] +=
12                $qteProduit ;
13        }
14        else
15        {
16            array_push( $_SESSION['panier']['libelleProduit'],$libelleProduit);
17            array_push( $_SESSION['panier']['qteProduit'],$qteProduit);
18            array_push( $_SESSION['panier']['prixProduit'],$prixProduit);
19        }
20    }
21    else
22        echo "Un problème est survenu, contactez l'administrateur du site.";
23 }
```

À la ligne 3, l'existence du panier est vérifié via la fonction précédente `creationPanier()` et il faut s'assurer que le panier n'est pas verrouillé. Si l'article existe déjà dans le panier (ligne 7) sa quantité dans le panier est augmenté sinon le produit est ajouté au panier.

Exemple 16 : Suppression d'un article

```
1 function supprimerArticle($libelleProduit){
2     if (creationPanier() && !isVerrouille())
3     {
4         $tmp=array();
5         $tmp['libelleProduit'] = array();
6         $tmp['qteProduit'] = array();
7         $tmp['prixProduit'] = array();
8         $tmp['verrou'] = $_SESSION['panier']['verrou'];
9
10        for($i = 0; $i < count($_SESSION['panier']['libelleProduit']); $i++)
11        {
12            if ($_SESSION['panier']['libelleProduit'][$i] !==
13                $libelleProduit)
14            {
15                array_push( $tmp['libelleProduit'],$_SESSION['panier']
16                    ['libelleProduit'][$i]);
17                array_push( $tmp['qteProduit'],$_SESSION['panier']
18                    ['qteProduit'][$i]);
19                array_push( $tmp['prixProduit'],$_SESSION['panier']
20                    ['prixProduit'][$i]);
21            }
22        }
23    }
24 }
```

```

22
23     }
24     $_SESSION['panier'] = $tmp;
25     unset($tmp);
26 }
27 else
28     echo "Un problème est survenu, contactez l'administrateur du site.";
29 }

```

Aux lignes 6 à 10, un panier d'achat temporaire est créé. La boucle for (ligne 10) permet de remplir le panier temporaire sans l'article à supprimer. Puis le panier de la session est remplacé par le panier temporaire (ligne 24).

Exemple 17 : Modification de la quantité d'un article

```

1  function modifierQteArticle($libelleProduit,$qteProduit){
2      //Si le panier existe
3      if (creationPanier() && !isVerrouille())
4      {
5          if ($qteProduit > 0)
6          {
7              //Recherche du produit dans le panier
8              $positionProduit = array_search($libelleProduit,
9                                              $_SESSION['panier']['libelleProduit']);
10
11              if ($positionProduit !== false)
12              {
13                  $_SESSION['panier']['qteProduit'][$positionProduit] =
14                                                              $qteProduit ;
15              }
16          }
17          else
18              supprimerArticle($libelleProduit);
19      }
20      else
21          echo "Un problème est survenu, contactez l'administrateur du site.";
22  }

```

Si la quantité demandée pour un produit est supérieure à 0 la quantité est modifiée. Si la quantité est négative ou nulle on supprime l'article à l'aide de la fonction `supprimerArticle()`.

Références

- Le PHP facile, <http://www.lephpfacile.com>
- Wikipédia, <https://fr.wikipedia.org/>
- W3Schools, XML Tutorial, <http://www.w3schools.com/>

- Tutoriel sur la création d'un panier en PHP, <http://jcrozier.developpez.com/articles/web/panier/>